# Part I: The Synchronous Baseline and Limits

# Original Tonic Send Path

## The Layer Sequence

1. **Application message** is created.
2. **Prost** encodes the protobuf body.
3. **Tonic** prepends the gRPC frame header.
4. **HTTP/2** transfers the frame bytes.

## Tonic's Classical API

```
// Synchronous execution on the CPU thread
fn encode(
    &mut self,
    item: Self::Item,
    dst: &mut EncodeBuf
) -> Result<(), Self::Error>;
```

*This is the path we start from before introducing accelerator copies.*

# Where the Copy Cost Appears

## Inside Prost Encode

- **Write tags and lengths**: Tiny varint writes (very cheap, fits in CPU registers).
- **Copy payload bytes**: Large contiguous memory copies (very expensive, CPU stalls on bus).

## Prost Code Pattern

```
// CPU writes tags/lengths directly,
// then performs costly memory copy:
pb_write_tag(dst, tag)?;
pb_write_varint(dst, payload.len())?;
dst.put_slice(&payload); // <- CPU block copy
```

We target the payload memory copy, leaving structured field writes to the CPU.

# Async Means Multiplexing

## Blocking vs Multiplexing

- **Blocking:** CPU thread is idle, locked on hardware completion.
- **Multiplexing:** CPU submits copies and switches immediately.

## Submission & Poll Logic

```
// Async offload pattern:
let fut = dsa_device.submit_copy(src, dst);
// CPU thread is now free to poll other tasks!
```

Async encode is a multiplexing mechanism to keep accelerator lanes fully occupied.

# Part II: Multiplexing and Thread Re-use

# How Async Encode Multiplexes Work

## 1. Start Work

Submit payload copy to DSA/IAX. Tonic moves the message and output buffer into one owned encode operation.

## 2. Pending

This stream cannot finish now, so it returns `Pending` to give the runtime thread permission to poll other work.

## 3. Resume

When the hardware completes, the waker is notified. Tonic polls the operation again and finishes the gRPC frame.

When encode returns `Pending`, the executor runtime thread is released immediately to multiplex other ready operations.

# Rust Async Multiplexes One Thread

## Cooperative Scheduling Loop

1. Poll encode future A → returns `Pending`.
2. Runtime thread runs ready operations B or C.
3. Hardware completion interrupt fires for A.
4. Runtime thread polls future A again → returns `Ready`.

## Tonic Stream Implementation

- `Encoder::Encode` is one message's encode future.
- `Pending` parks the message's copy work.
- `EncodedBytes.in_flight` stores the parked future.
- `Ready(EncodeBuffer)` returns completed bytes to Tonic.

Rust async multiplexes a thread: park the blocked encode, run ready operations, then resume.

# Part III: Memory Safety and Ownership

# What is a Rust Future?

## Core Trait Definition

A `Future` represents an asynchronous computation that eventually completes.

```
pub trait Future {
    type Output;

    // Polled repeatedly by the executor
    fn poll(
        self: Pin<&mut Self>,
        cx: &mut Context<'_>,
    ) -> Poll<Self::Output>;
}
```

## State Machine Mechanics

- **Poll-Based**: Futures are lazy; they make no progress unless the executor polls them.
- **Poll::Ready(val)**: Computation is complete; returns the final output.
- **Poll::Pending**: Computation is blocked (e.g. waiting for hardware DMA). Control yields.
- **Waker Notification**: When the event finishes, the executor is notified to poll again.

A Rust Future is a state machine: it starts, advances on poll, and yields control when waiting for hardware.

# Why Async Forces Ownership: The Multiplexing Reality

## The Multiplexing Requirement

To keep slow hardware lanes fully occupied, a single CPU thread must switch between many concurrent requests.

- The encode future **cannot** live on the caller's stack frame.
- It must be stored in the driver to survive suspension.
- This forces a `'static` lifetime bound on the future.

## The Compiler Logic

```rust
// 1. Sync Signature: Borrowed Stack Buffer
fn encode(
  &mut self,
  item: Item,
  dst: &mut EncodeBuf
) -> Result<(), Error>;

// 2. Async Signature: Owned Buffer & Future
fn encode(
  self: Pin<&mut Self>,
  item: Self::Item,
  dst: EncodeBuffer,
) -> Result<Self::Encode, Self::Error>;
// where Self::Encode: Future + 'static
```

To park a request and multiplex the thread, the future must own its buffer and message.

# Storing the Resumable State

## State Storage

The driver stores the owned future in `EncodedBytes::in_flight`.

The stack-frame is long gone, so the future keeps all data, progress, and wakers alive.

## Owned Future Fields

```rust
pub struct DsaAsyncProstEncode<T> {
    // Owned state must survive .await:
    state: PollEncodeState<DsaSink>,
    item: Option<T>,
    sink: Option<DsaSink>,
    stage: DsaEncodeStage,
}
```

- The future must own all data compartments to safely survive suspension crossing `.await`.

# Synthesis: How it Works

## 1. Driver Poll Loop

Tonic driver polls the future. If `Pending`, the thread is released. When completion fires, the waker schedules another poll.

## 2. Prost Resumable State

Prost checks the phase index to avoid rewriting tag/length metadata when resuming.

```
if state.phase() == 0 {
    pb_write_tag(dst, tag)?;
    pb_write_varint(dst, len)?;
    state.set_phase(1); // Set checkpoint!
}
// Resumes here if phase == 1:
poll_copy_payload(dst, state, cx)
```

- Async encode = owned resumable state + exact protocol resume points.

# Part IV: Turning Prost Async: Resumable Serialization

# How Prost Generates Code

## 1. Protobuf Schema (.proto)

Prost parses the `.proto` schema to define message layouts and field metadata.

```
syntax = "proto3";

message UserProfile {
    string name = 1;
    bytes avatar = 2;
}
```

## 2. Generated Rust Struct

`prost-build` compiles it into a standard Rust struct with serialization attributes.

```
#[derive(Clone, PartialEq, ::prost::Message)]
pub struct UserProfile {
    #[prost(string, tag = "1")]
    pub name: ::prost::alloc::string::String,
    #[prost(bytes = "vec", tag = "2")]
    pub avatar: ::prost::alloc::vec::Vec<u8>,
}
```

> Prost compiles schemas into standard Rust structs. To change serialization behavior, we customize the code generated for these structs.

# Sync Serialization: Sequential Path

## Generated Sync Code

The standard macro derive generates a simple sequential writer.

```
// Emitted by #[derive(prost::Message)]
fn encode_raw(
    &self,
    buf: &mut impl BufMut
) {
    // field 1: string name
    string::encode(1, &self.name, buf);

    // field 2: bytes avatar
    bytes::encode(2, &self.avatar, buf);
}
```

## Execution Details

- **Sequential execution**: All fields are processed in order on the calling CPU thread.
- **No suspension**: The execution is continuous and cannot yield back to the caller.
- **Synchronous copies**: Large payload copies (`dst.put_slice`) run synchronously, blocking the CPU.

Simple and fast for CPU-only execution, but blocks the thread during large payload memory copies.

# Async Serialization: Resumable Path

## Generated Async Code

Custom codegen emits a resumable state-machine encoder.

```
fn poll_encode_raw<S>(&self, sink, state, cx) -> Poll<Result<(),
Error>> {
    if state.field() == 0 {       // string
        ready!(string::poll_encode(1, &self.name, sink, state,
cx))?;
        state.advance_field();
    }
    if state.field() == 1 {       // bytes
        ready!(bytes::poll_encode(2, &self.avatar, sink, state,
cx))?;
        state.advance_field();
    }
    state.clear();
    Poll::Ready(Ok(()))
}
```

## Execution Details

- **Flat field-indexed loop**: Guarded by `state.field()` cursor to remember position.
- **Yield on Pending**: If `poll_encode` returns `Pending`, execution suspends.
- **Resume on Wake**: Execution resumes directly at the suspended field, avoiding repeating previous writes.

`String` and `Bytes` fields use `poll_encode` (may yield); other scalar types write directly without suspending.

# Split-Phase Serialization: CPU vs Async

## CPU Structure Phase

Protobuf metadata and scalar fields are written synchronously by the CPU directly to the mutable buffer.

- **Tags and wire types**: Compact varints (1-5 bytes).
- **Length delimiters**: Size prefixes for nested messages, strings, and bytes.
- **Scalars**: Integers, floats, and booleans.

:

## Offloaded Payload Phase

Large contiguous byte arrays and strings are offloaded asynchronously, allowing the CPU thread to yield.

- **Dynamic selection**: Small payloads fall back to CPU memory copies immediately.
- **Async copy**: Large payloads submit to the DSA/IAX work queue and return `Pending`.

Protobuf encoding is split: CPU writes structure synchronously, while hardware copies payloads asynchronously.

# State Tracking & Hierarchical Nesting

## State Cursor & Nesting Stack

To resume serialization at the exact byte boundary after a `Pending` yield, `PollEncodeState` tracks:

- **field**: The index of the current field in the message.
- **index**: The iteration index for repeated/packed fields.
- **phase**: The execution checkpoint within a single field.
- **nested**: A frame stack tracking parent message contexts during recursion.

:

## Nesting and Recursion State

```rust
pub struct PollEncodeState<S: AsyncEncodeTarget> {
    root: PollEncodeFrame,        // Current message cursor
    nested: Vec<PollEncodeFrame>, // Stack for parent messages
    depth: usize,                 // Current depth level
    payload: S::PayloadState,     // Hardware copy state
}

struct PollEncodeFrame {
    field: usize,
    index: usize,
    phase: u8,
}
```

Hierarchical messages enter a nested state: push parent frame, poll sub-message, and pop on completion.

# The Generated Resumable State Machine

## Resume Execution Flow

The generated `poll_encode_raw` uses a flat state machine:

- **Branch to Field**: Select field via `state.field()`.
- **Phase Checkpoint**: Check `state.phase()`. If `0`, perform CPU metadata writes and advance phase to `1`.
- **Yield/Resume**: Call `poll_write_payload`. If `Pending`, yield. On wakeup, resume directly at phase `1` (skipping metadata writes).

## Generated Code Pattern

```
if state.field() == 0 { // string field
    if state.phase() == 0 {
        let mut buf = sink.buf_mut();
        encode_key(1, WireType::LengthDelimited, &mut buf);
        encode_varint(self.name.len() as u64, &mut buf);
        state.set_phase(1);
    }
    match sink.poll_write_payload(self.name.as_bytes(),
                                  state.payload_pin_mut(), cx) {
        Poll::Ready(Ok(())) => {
            state.set_phase(0);
            state.advance_field();
        }
        Poll::Pending => return Poll::Pending,
    }
}
```

Phase transitions ensure that metadata prefixes are written exactly once, even across multiple yields.

# Reusable Systems Rule

**Thread-multiplexing with hardware offloads requires owned resumable state.**

### Tonic Frame

Stream driver owns header reservation, compression, size check, and final gRPC framing.

### Encode Future

Suspended operation owns message, `EncodeBuffer`, offload state, and drop/cancel cleanup.

### Prost Wire Checkpoint

Inner serialization engine tracks field, index, phase, and sub-message recursion frames.

**Performance Balance Check:** Benchmarks decide whether the offload wins: does avoided CPU staging outweigh submission and bookkeeping overhead?